

Galileo: Towards Performance Robustness for Microservices

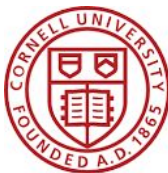
Authors: Diyanstu Saxena, Gaurav Vipat, Jiaxin Lin, Jingbo Wang,
Isil Dilig, Sanjay Shakkottai, Aditya Akella

Presented by: Sanika Nandpure



TEXAS

The University of Texas at Austin



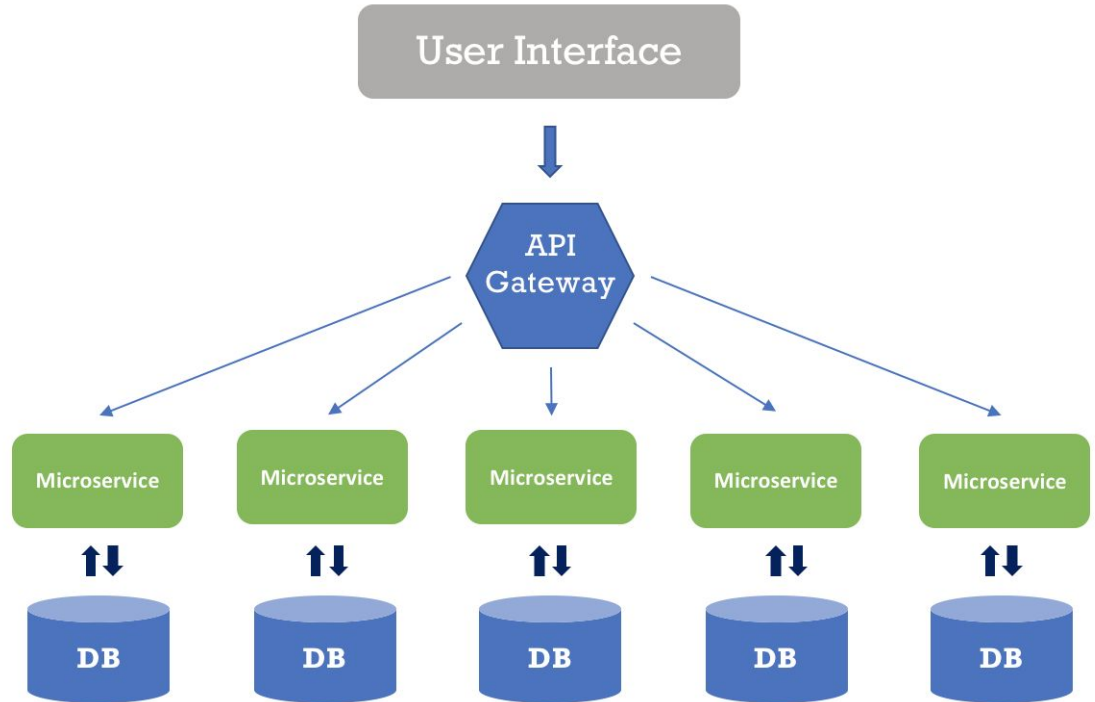
Cornell University



PURDUE
UNIVERSITY®

Microservices

- Independently deployed, loosely coupled
- Each service tackles a narrow task
- In large microservice applications, latency behavior can be quite unpredictable



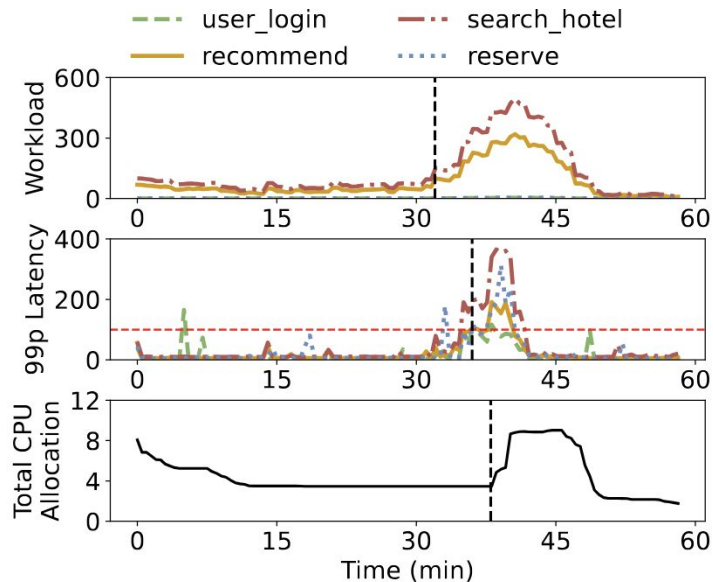
Microservice controllers

- Controllers include:
 - Autoscalers
 - Admission controllers
- Can be heuristic-based or learned
- Autothrottle:
 - Online RL-based (contextual bandit algorithm)
 - At each step, observe current latencies and request rates
 - If latency within SLO, minimize resource usage
 - If latency not within SLO, minimize latency
- TopFull
 - Offline trained using PPO
 - Observes current goodput and latencies, and decides rate limit for each request type

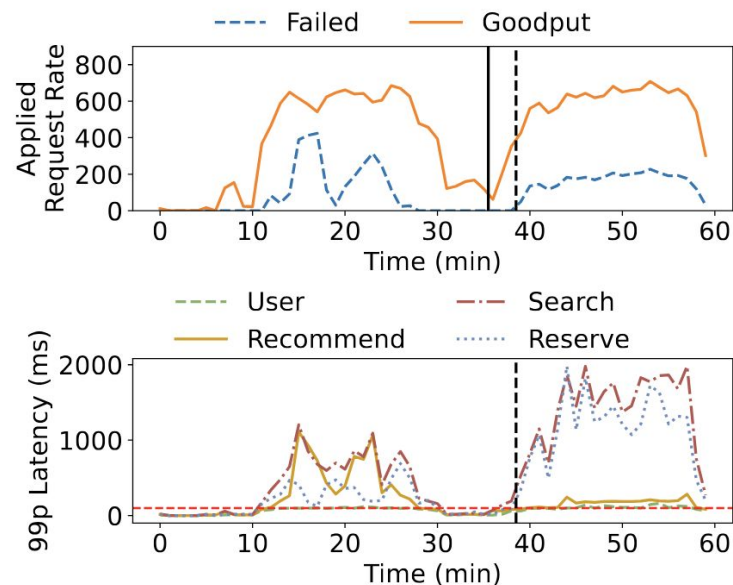
Microservice latency behavior - Motivation studies

- Microservices have unpredictable behavior under bursty workloads
 - Stable regime: processing capacity is higher than arrival rate $\Rightarrow$ **latencies are predictable**
 - Overloaded regime: arrivals outpace capacity $\Rightarrow$ **queues grow, causing increased latency (unpredictable)**
 - Unpredictable due to dynamically changing workloads and resource contention between co-running jobs

How well are current controllers able to deal with unpredictable workloads?



Autothrottle

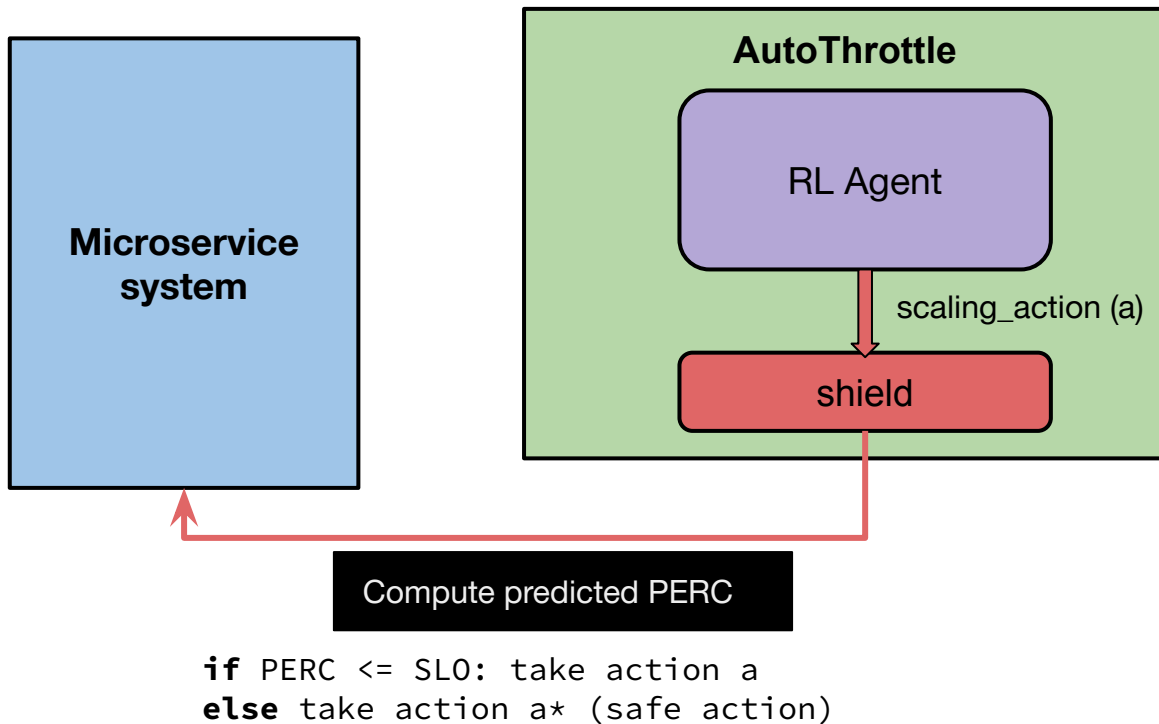


TopFull

Key insights from motivation experiments:

- Changes to input workloads can drastically inflate tail latency
- Currently, controllers are **reactive**, causing suboptimal behavior on dynamic/bursty workloads
- Controllers need to proactively consider this dynamic aspect

Galileo: High-level solution (autoscaler)



PERC (Performance Robustness Certificates)

- PERC(ϵ , E, T)
 - $\epsilon \rightarrow$ target percentile
 - E $\rightarrow$ environment
 - T $\rightarrow$ request type
- PERC(ϵ , E, T) = maximum ϵ th percentile end-to-end latency of a request type T under an environment E, given some fixed set of possible perturbations that can be made to the system

$$P_{\epsilon,E,T} = \max\{l_{\epsilon,T}(E') \mid E' \in \Delta_E\}$$

Provides reliable estimate of latency upper bound of a request.

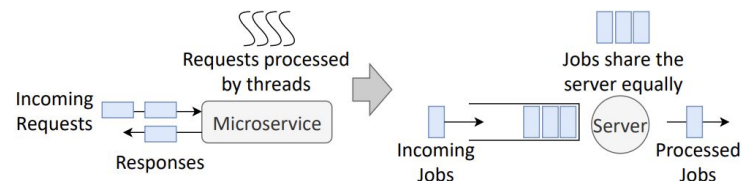
Challenges in implementing PERCs

- #1: How do we model end-to-end latency of the system?
- #2: How do we define the perturbation set?
- #3: How do we map perturbation to end-to-end performance impact?
- #4: How do we fit the model to live applications?
- #5: How do we integrate PERC with a microservice controller?

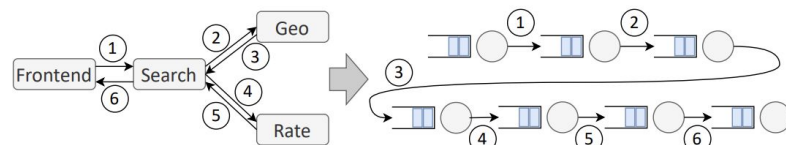
Challenge #1: How do we model end-to-end latency?

- **Solution:** Performance Reasoning Model (PRM)
 - Each service is represented as an independent mathematical model, parameterized by arrival and processing rates

Each microservice is modeled as a $M/M/1$ -PS queue.



(a) A single microservice modeled as an $M/M/1$ -PS queue.



(b) Services invoked by a user request modeled as a network of $M/M/1$ -PS queues, where requests may re-enter other queues. Here, geo and rate are invoked sequentially.

Figure 5: Modeling the latency of a user request in a microservice applications using a network of $M/M/1$ -PS queues.

Queuing theory

Studies how a queue is formed and how it behaves over time.

Kendall's Notation

This is a way to describe different types of queuing systems. David George Kendall introduced it, and it uses a simple format:

A/S/c/K/N/D, where:

- **A** for the arrival process.
- **S** for the mathematical distribution of the service time.
- **c** for the number of servers.
- **K** for the capacity of the queue (if not unlimited).
- **N** for the number of possible customers (if not infinite).
- **D** for the queuing discipline (first-in, first-out by default).

In most real-world cases, you only need the first three: **A/S/c**.

M / M / 1-PS

(1) (2) (3)

(1) Distribution of request arrivals

Markovian/memoryless
Poisson

$$P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

(2) Distribution of service time

Markovian/memoryless
Exponential

$$f(x) = \mu e^{-\mu x}$$

(3) Num servers

1 server w/ processor sharing

Performance Reasoning Model (PRM)

- $M / M / 1$ -PS is an exact model for thread-per-request implementations
- For fixed thread pools, is an approximation given pool size is sufficiently large

This way of mathematically modeling services allows for closed-form expressions for end-to-end latency.

Closed-form end-to-end latency

Sojourn time = total time a request spends in a service from arrival to completion = (execution + waiting time)

If a request goes through services (1, 2, ... K), where each service is modeled as a queue, then the sojourn time at queue i is:

$$\begin{aligned}\theta_i &\sim \text{Exp}(\gamma_i) & \mu_i &= \text{service time parameter} \\ \gamma_i &= (\mu_i - \lambda_i) & \lambda_i &= \text{arrival rate parameter}\end{aligned}$$

Theorem: Assuming that jobs are processed in the order they arrive, and that every job completes in finite time, the sojourn time for each queue is independent and exponentially distributed.

Closed-form end-to-end latency

Given this theorem, the end-to-end sojourn time of a request will be given as: $\theta = \sum_i \theta_i$

The sum of exponentials can be approximated as a Gamma distribution:

$$\theta \sim \Gamma(\alpha, \beta); \quad \alpha = \frac{(\sum \gamma_i)^2}{\sum \gamma_i^2}, \quad \beta = \frac{\sum \gamma_i^2}{\sum \gamma_i}$$

Then the ϵ th percentile latency can be calculated as:

$$l_{\epsilon, T}(E) = ppf(\Gamma(\alpha, \beta), \epsilon)$$

Which tells us the ϵ th percentile value for end-to-end sojourn time of a request.

Performance Reasoning Model (PRM)

Different request types may invoke a different network of services, so we create a separate PRM for each request type r .

Thus, PRM is defined as:

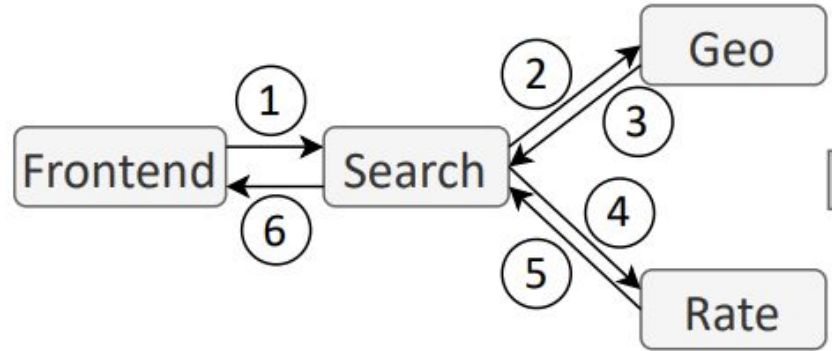
$$\mathbf{M}_r = \mathbf{S}_r \mathbf{X} \mathbf{D}_r$$

$\mathbf{S}_r$ = service parameters of all queues invoked by the request type r (μ_i, λ_i)

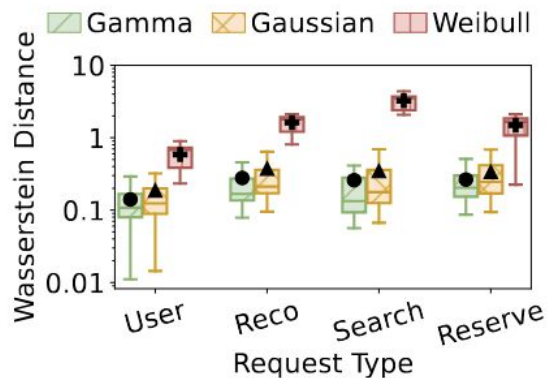
$\mathbf{D}_r$ = end-to-end distribution parameters of the network invoked by request type r (α, β)

PRM - Dealing with concurrent requests

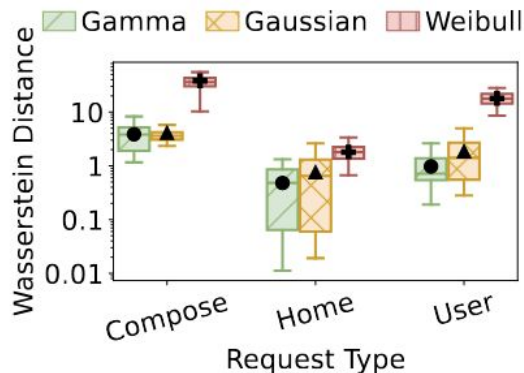
End-to-end latency of a request = **max**(latencies of execution paths)



Empirical evaluation



(a) Hotel Reservation



(b) Social Network

Figure 6: Wasserstein distances of proposed Gamma model vs. baselines. Y-axis is in log scale; markers show means.

- Gamma's Wasserstein distance:
 - 5-44% better than Gaussian
 - 73-95% better than Weibull
- Ratio of tail latencies of the fitted distribution to full set of collected latency samples:
 - 0.83-1.01x ground truth p99 on average

Challenge #2: How do we define the perturbation set?

$$P_{\epsilon, E, T} = \max \{ l_{\epsilon, T}(E') \mid E' \in \Delta_E \}$$

PRM
approximation
gives us this

Now we need to figure out
set of valid “perturbed”
environments that could
realistically occur.

Challenge #2: How do we define the perturbation set?

We saw how PRM is basically just a set of service and end-to-end latency distribution parameters.

Perturbation is just a change η to the arrival/processing rates of one or more services.

$$\begin{aligned} P &::= \text{perturb}_{\mu} \ i \ \eta \mid \text{perturb}_{\lambda} \ i \ \eta \\ \text{actions} &::= P \mid P \circ P \end{aligned}$$

What does a “change of η ”
mean?

$$\begin{aligned} \forall i, \eta. \llbracket \text{perturb}_{\mu} \ i \ \eta \rrbracket (\mathcal{M}_r) = & \mathcal{M}_r[\mu_i \mapsto \mu_i + \eta \mu_i, \\ & \alpha \mapsto \alpha + \mathcal{P}_{\alpha}(\mu_i, \eta), \\ & \beta \mapsto \beta + \mathcal{P}_{\beta}(\mu_i, \eta)] \end{aligned}$$

Defining range of perturbations

Finally, to make make the set ΔE of perturbations:

$$\Delta_E = \left\{ \bigcirc_i \llbracket (\text{perturb}_\mu \ i \ \eta_{\mu_i}) \circ (\text{perturb}_\lambda \ i \ \eta_{\lambda_i}) \rrbracket (\mathcal{M}_r) \right. \\ \left. \forall i \in N(r), -\delta < \eta_{\mu_i}, \eta_{\lambda_i} < \delta \right\}$$

- $N(r)$ = set of services invoked
- δ = hyperparameter
large δ means wider perturbation set, small means narrower set

Challenge #3: How do we map perturbation to end-to-end performance impact?

$$P_{\epsilon, E, T} = \max\{l_{\epsilon, T}(E') \mid E' \in \Delta_E\}$$

Challenge #3: How do we map perturbation to end-to-end performance impact?

$$P_{\epsilon, E, T} = \max\{l_{\epsilon, T}(E') \mid E' \in \Delta_E\}$$

We don't actually need to calculate latencies
for all possible perturbation values in the set!

Challenge #3: How do we map perturbation to end-to-end performance impact?

Ultimately, to calculate PERC we only need to know latency under **worst case** perturbation, which occurs when:

- Processing rates at all queues decrease by maximum value possible ($-\delta$)
- Arrival rates at all queues increase by the maximum value possible (δ)

Challenge #3: How do we map perturbation to end-to-end performance impact?

Ultimately, to calculate PERC we only need to know latency under **worst case** perturbation, which occurs when:

- Processing rates at all queues decrease by maximum value possible ($-\delta$)
- Arrival rates at all queues increase by the maximum value possible (δ)

To calculate PERC, we only need to calculate end-to-end latency for this particular case:

$$\forall i \in N(r), \bigcirc_i [(\text{perturb}_\mu i - \delta) \circ (\text{perturb}_\lambda i \delta)](\mathcal{M}_r)$$

To do this, we need: latency distribution parameters (α, β) of current environment E , and the perturbation functions P_α and P_β .

How do we get these parameters?

Ideally if we know the service parameters (μ, λ) of each service, we can easily calculate the current distribution parameters and perturbation functions from the equations we saw previously

$$\theta \sim \Gamma(\alpha, \beta); \quad \alpha = \frac{(\Sigma \gamma_i)^2}{\Sigma \gamma_i^2}, \quad \beta = \frac{\Sigma \gamma_i^2}{\Sigma \gamma_i}$$

$$\begin{aligned} \forall i, \eta. \llbracket \text{perturb}_\mu i \ \eta \rrbracket (\mathcal{M}_r) = & \mathcal{M}_r[\mu_i \mapsto \mu_i + \eta \mu_i, \\ & \alpha \mapsto \alpha + \mathcal{P}_\alpha(\mu_i, \eta), \\ & \beta \mapsto \beta + \mathcal{P}_\beta(\mu_i, \eta)] \end{aligned}$$

How do we get these parameters?

Ideally if we know the service parameters (μ, λ) of each service, we can easily calculate the current distribution parameters and perturbation functions from the equations we saw previously

$$\theta \sim \Gamma(\alpha, \beta); \quad \alpha = \frac{(\Sigma \gamma_i)^2}{\Sigma \gamma_i^2}, \quad \beta = \frac{\Sigma \gamma_i^2}{\Sigma \gamma_i}$$
$$\forall i, \eta. \llbracket \text{perturb}_\mu i \ \eta \rrbracket (\mathcal{M}_r) = \mathcal{M}_r[\mu_i \mapsto \mu_i + \eta \mu_i, \\ \alpha \mapsto \alpha + \mathcal{P}_\alpha(\mu_i, \eta), \\ \beta \mapsto \beta + \mathcal{P}_\beta(\mu_i, \eta)]$$

In reality, actually getting these values is not simple.

- Instrumenting adds significant overhead
- Microservices are dynamic, and can lead to noisy measurements

Solution: Infer these parameters from end-to-end latency alone

- **Step 1:** At short, fixed intervals, fit a Gamma distribution to recent end-to-end latencies for each request type $\Rightarrow$ this gets us α , β
- **Step 2:** To get P_α and P_β , use gradient estimation methods

$$\alpha = f_\alpha(\mu_1, \mu_2, \dots, \lambda_1, \lambda_2, \dots)$$

$$\mathcal{P}_\alpha(\mu_i, \eta) = \eta \mu_i \frac{\partial f_\alpha}{\partial \mu_i} \approx \eta \mathcal{K}_{\alpha, \mu_i}$$

Key idea: the value of f_α at μ_i is known from Step 1. To get value at $\mu_i + (\eta_i)(\mu_i)$, we can fit another Gamma distribution over end-to-end latencies collected with processing rate μ_i , manually tweaked by η .

Data-efficient way to approximate parameters!

Solution: Infer these parameters from end-to-end latency alone

- **Step 1:** At short, fixed intervals, fit a Gamma distribution to recent end-to-end latencies for each request type $\Rightarrow$ this gets us α , β
- **Step 2:** To get P_α and P_β , use gradient estimation methods

$$\alpha = f_\alpha(\mu_1, \mu_2, \dots, \lambda_1, \lambda_2, \dots)$$

$$\mathcal{P}_\alpha(\mu_i, \eta) = \eta \mu_i \frac{\partial f_\alpha}{\partial \mu_i} \approx \eta \mathcal{K}_{\alpha, \mu_i}$$

Key idea: the value of f_α at μ_i is known from Step 1. To get value at $\mu_i + (\eta_i)(\mu_i)$, we can fit another Gamma distribution over end-to-end latencies collected with processing rate μ_i , perturbed by η .

Data-efficient way to approximate parameters!

Also deals with **Challenge #4** (how to fit model to live applications)

In practice:

- Fit Gamma distribution to latency samples from past 60 seconds
- Apply $N = 10$ small perturbations
- Collect new samples for 20 seconds after each perturbation to estimate updated parameters
- Gradient estimation cycle takes ~210 seconds

Finally, calculate PERC:

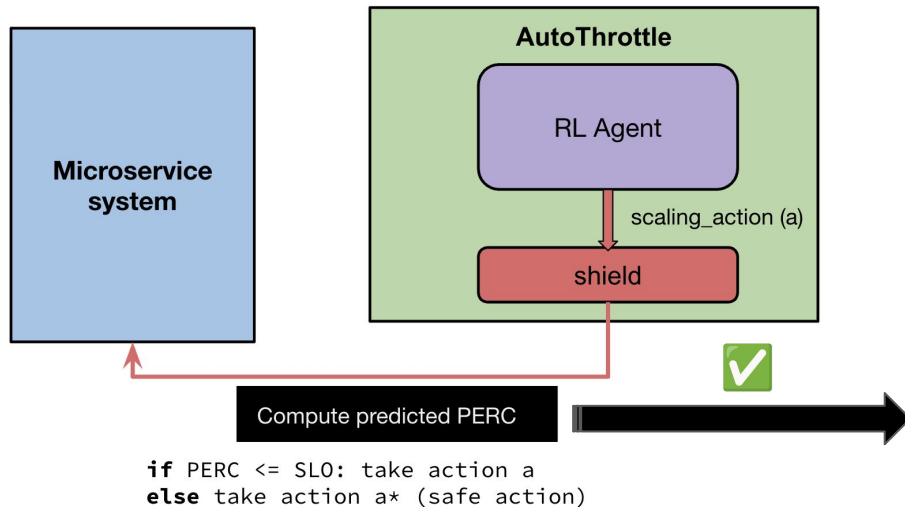
$$\alpha^* \mapsto \alpha + \sum_i \mathcal{P}_\alpha(\mu_i, -\delta) + \sum_i \mathcal{P}_\alpha(\lambda_i, \delta) \quad (9)$$

$$\beta^* \mapsto \beta + \sum_i \mathcal{P}_\beta(\mu_i, -\delta) + \sum_i \mathcal{P}_\beta(\lambda_i, \delta) \quad (10)$$

The resulting PERC can now be computed using Equation (3):

$$P_{\epsilon, E, T} = ppf(\Gamma(\alpha^*, \beta^*), \epsilon) \quad (11)$$

Galileo: High-level solution (autoscaler)



if PERC $\leq$ SLO: take action a
else take action a^* (safe action)

Algorithm 1 PERC Computation

Input: Request type r ; tail percentile ϵ ; number of perturbation samples N ; current control (allocation/rate limits) $\mathbf{c}$; operator-specified perturbation budget δ .

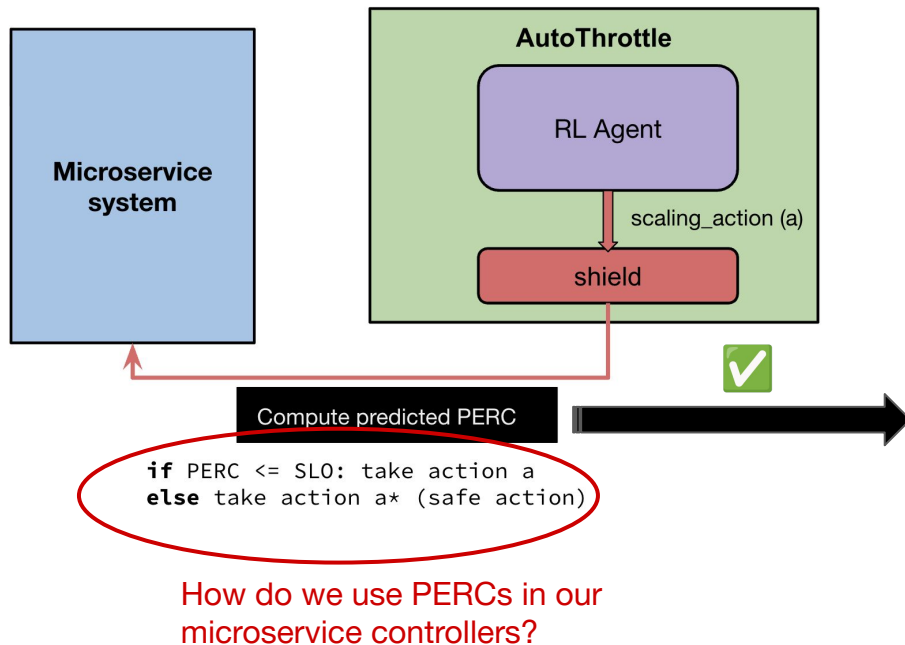
Output: Worst-case tail latency estimate $P_{\epsilon,E,T}$ (the PERC).

```

1: procedure COMPUTECERTIFICATE( $\epsilon, \mathbf{c}, r, \delta, N$ )
2:    $D \leftarrow \text{COLLECTLATENCIES}(r)$   $\triangleright$  Gather latencies for type  $r$ 
3:    $(\alpha, \beta) \leftarrow \text{GAMMAFIT}(D)$   $\triangleright$  Fit Gamma distribution
4:    $\mathcal{P}_\alpha, \mathcal{P}_\beta \leftarrow \text{COMPUTEGRAIENTS}(r, \mathbf{c}, N)$ 
    $\triangleright$  Use gradient estimation to compute perturbation functions.
5:    $\alpha^* \leftarrow \alpha + \sum_i \mathcal{P}_\alpha(\mu_i, -\delta) + \sum_i \mathcal{P}_\alpha(\lambda_i, \delta)$ 
6:    $\beta^* \leftarrow \beta + \sum_i \mathcal{P}_\beta(\mu_i, -\delta) + \sum_i \mathcal{P}_\beta(\lambda_i, \delta)$ 
    $\triangleright$  Worst-case distribution parameters using Equations (9) and (10)
7:    $P_{\epsilon,E,T} \leftarrow \text{ppf}(\Gamma(\alpha^*, \beta^*), \epsilon)$   $\triangleright$  Using Equation (3)
8:   return  $P_{\epsilon,E,T}$ 

9: procedure COMPUTEGRAIENTS( $\mathbf{c}, r, N$ )
10:   $\mathbf{A} \leftarrow \text{PERTURBATIONMATRIX}(N)$   $\triangleright$  See Algorithm 1 in [10]
11:  for  $j = 1 \rightarrow N$  do  $\triangleright$  Make  $N$  perturbations
12:     $\mathbf{c}^{(j)} \leftarrow \text{UPDATECONTROL}(\mathbf{c}, \mathbf{a}_j)$   $\triangleright$  Perturb control.
13:     $D^{(j)} \leftarrow \text{COLLECTLATENCIES}(r)$ 
14:     $(\alpha^{(j)}, \beta^{(j)}) \leftarrow \text{GAMMAFIT}(D^{(j)})$   $\triangleright$  Fit to updated latencies
15:     $(\widehat{\nabla}\alpha, \widehat{\nabla}\beta) \leftarrow \text{RECOVERGRADIENTS}(\{\alpha, \beta, (\alpha^{(j)}, \beta^{(j)})\}_{j=1}^N)$ 
    $\triangleright$  Estimate gradients via Algorithm 1 in [10]
16:  return  $(\widehat{\nabla}\alpha, \widehat{\nabla}\beta)$ 
    
```


Galileo: High-level solution (autoscaler)



Algorithm 1 PERC Computation

Input: Request type r ; tail percentile ϵ ; number of perturbation samples N ; current control (allocation/rate limits) $\mathbf{c}$; operator-specified perturbation budget δ .

Output: Worst-case tail latency estimate $P_{\epsilon,E,T}$ (the PERC).

```

1: procedure COMPUTECERTIFICATE( $\epsilon, \mathbf{c}, r, \delta, N$ )
2:    $D \leftarrow \text{COLLECTLATENCIES}(r)$   $\triangleright$  Gather latencies for type  $r$ 
3:    $(\alpha, \beta) \leftarrow \text{GAMMAFIT}(D)$   $\triangleright$  Fit Gamma distribution
4:    $\mathcal{P}_\alpha, \mathcal{P}_\beta \leftarrow \text{COMPUTEGRAIENTS}(r, \mathbf{c}, N)$ 
    $\triangleright$  Use gradient estimation to compute perturbation functions.
5:    $\alpha^* \leftarrow \alpha + \sum_i \mathcal{P}_\alpha(\mu_i, -\delta) + \sum_i \mathcal{P}_\alpha(\lambda_i, \delta)$ 
6:    $\beta^* \leftarrow \beta + \sum_i \mathcal{P}_\beta(\mu_i, -\delta) + \sum_i \mathcal{P}_\beta(\lambda_i, \delta)$ 
    $\triangleright$  Worst-case distribution parameters using Equations (9) and (10)
7:    $P_{\epsilon,E,T} \leftarrow \text{ppf}(\Gamma(\alpha^*, \beta^*), \epsilon)$   $\triangleright$  Using Equation (3)
8:   return  $P_{\epsilon,E,T}$ 

9: procedure COMPUTEGRAIENTS( $\mathbf{c}, r, N$ )
10:   $\mathbf{A} \leftarrow \text{PERTURBATIONMATRIX}(N)$   $\triangleright$  See Algorithm 1 in [10]
11:  for  $j = 1 \rightarrow N$  do  $\triangleright$  Make  $N$  perturbations
12:     $\mathbf{c}^{(j)} \leftarrow \text{UPDATECONTROL}(\mathbf{c}, \mathbf{a}_j)$   $\triangleright$  Perturb control.
13:     $D^{(j)} \leftarrow \text{COLLECTLATENCIES}(r)$ 
14:     $(\alpha^{(j)}, \beta^{(j)}) \leftarrow \text{GAMMAFIT}(D^{(j)})$   $\triangleright$  Fit to updated latencies
15:     $(\hat{\nabla}\alpha, \hat{\nabla}\beta) \leftarrow \text{RECOVERGRADIENTS}(\{\alpha, \beta, (\alpha^{(j)}, \beta^{(j)})\}_{j=1}^N)$ 
    $\triangleright$  Estimate gradients via Algorithm 1 in [10]
16:  return  $(\hat{\nabla}\alpha, \hat{\nabla}\beta)$ 
    
```

Challenge #5: How do we integrate PERC w/ controller?

- Shielding = safe RL technique
 - Can be used during both training and inference
 - Monitor agent actions; if the agent is taking some unsafe action, substitute agent's action with a safe backup action
 - During training: if shield is triggered, agent also incurs a large negative penalty; incentivizes agent to take actions that meet property P
- Shield = (M, P, Safe)
 - M: system model
 - P: property of interest
 - Safe: function that takes in scaling action a and outputs provably safe action a^*
- Shield is triggered only when scaling action **a violates P given model M**

How do we define an action a ?

- Controller actions include things like adjusting resource allocations, rate limits, etc.
- Just like perturbations, can be represented as updates to service parameters in PRM
- Concretely:
 - Changing CPU resource allocations for service i changes processing rate μ
 - Changing rate limit for request type r changes arrival rates λ_i for all services i in $N(r)$.

$\text{controls} ::= \text{adjustCPU } i \ \eta \mid \text{rateLimit } r \ \eta$

$$\llbracket \text{adjustCPU } i \ \eta \rrbracket(\mathcal{M}_r) = \llbracket \text{perturb}_\mu i \ \eta \rrbracket(\mathcal{M}_r)$$

$$\llbracket \text{rateLimit } r \ \eta \rrbracket(\mathcal{M}_r) = \bigcirc_{i \in N(r)} \llbracket \text{perturb}_\lambda i \ \eta \rrbracket(\mathcal{M}_r)$$

How do we check whether a meets SLO?

- Calculate worst case perturbations to the environment after we take scaling action a.
- Compute PERC
- Check: Is this predicted PERC $\leq$ SLO?
 - If yes: take scaling action a
 - If no: trigger shield

Computing safe action a^*

- Computing a^* :
 - Insight: end-to-end latencies are monotonic wrt arrival and processing rates
 - Higher arrival rates $\rightarrow$ higher latencies
 - Higher processing rates $\rightarrow$ lower latencies
 - We can compute a^* by repeatedly increment or decrementing a along the direction of the gradients of the perturbation functions until we are within SLO requirement

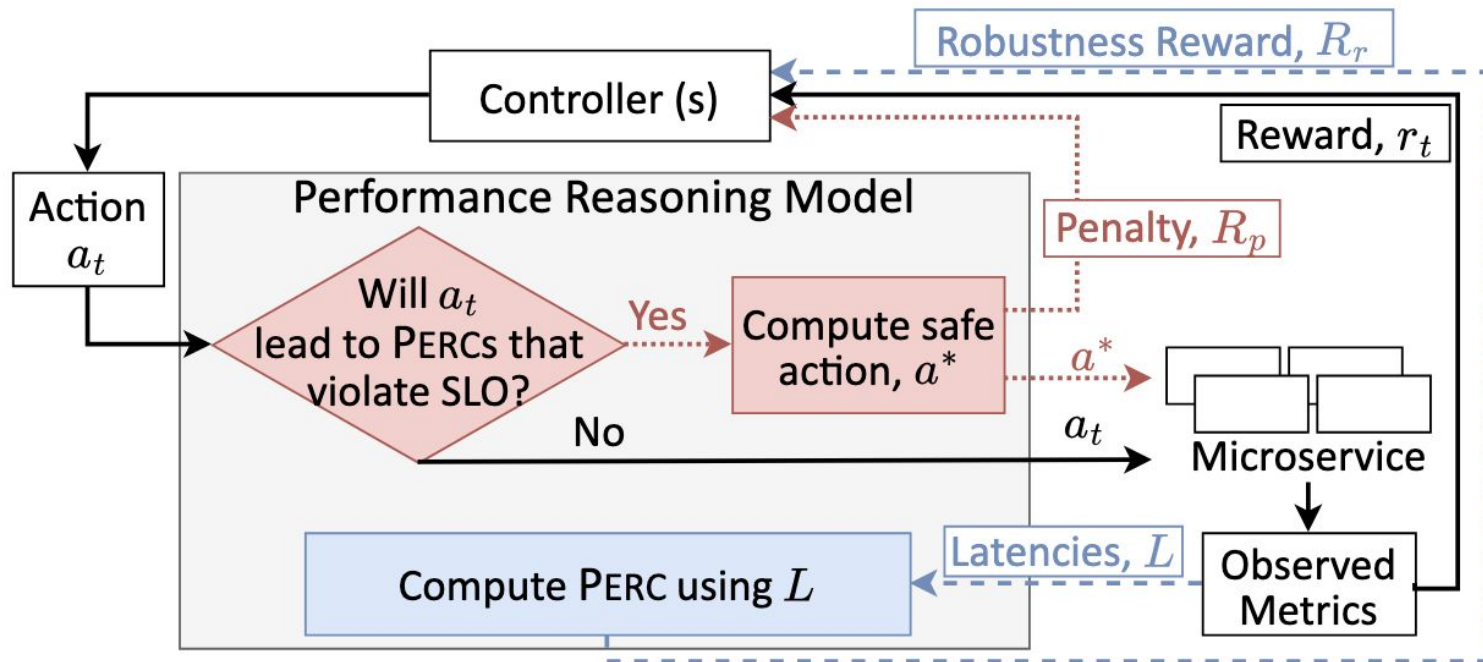
Note: During training

- Can use shield during training as part of reward function

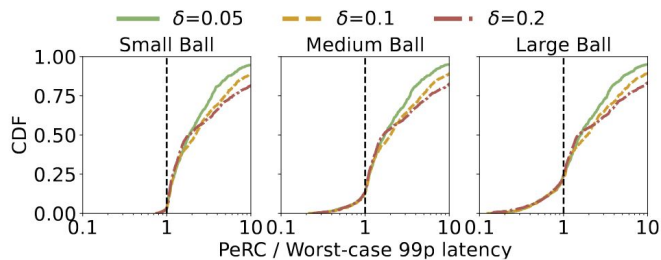
$$r = R_o + wR_r + R_p,$$

- R_o : base reward
- R_p : large negative penalty if agent's proposed action (a) would violate SLO
- R_r : sigmoid(PERC - SLO)
 - Helps with sparse reward from R_p
 - Allows the agent to detect bad actions proactively

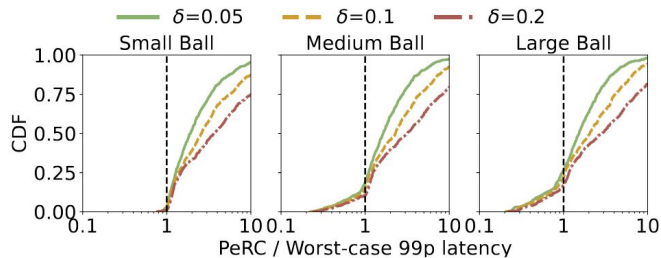
All together: Galileo



Evaluation: PERCs



(a) HR: search Request Type



(b) SN: compose Request Type

Figure 8: Tightness of PERCs: CDF of the ratio of the computed PERC in an environment to the worst-case 99p latency under a perturbation ball. Curves close to the vertical line at 1 imply tight bounds, to the left of the vertical line imply weak bounds, and to the right of the vertical line imply a loose upper bound.

Setup

- DeathStarBench HotelReservation, SocialNetwork
- 250 hours
- 0-1500 rps
- Background jobs consuming 40-80% of CPUs

Evaluation: create Small, Medium, and Large neighborhoods around environment E, compute PERC @ E and compare against p99 latencies observed across each neighborhood level.

Metric:

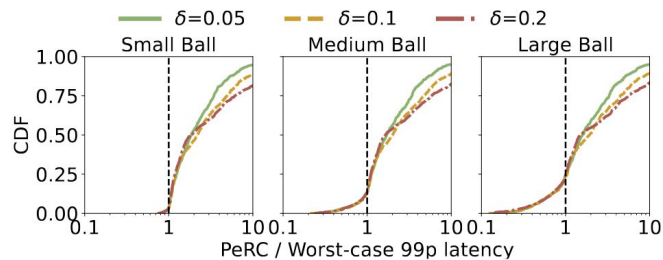
$$Tightness_{T,\delta}(E) = \frac{P_{\epsilon,E,T}}{\max\{l_{\epsilon,T}(E') \mid E' \in \Delta_{E,\delta}\}}$$

= 1 $\Rightarrow$ perfect PERC

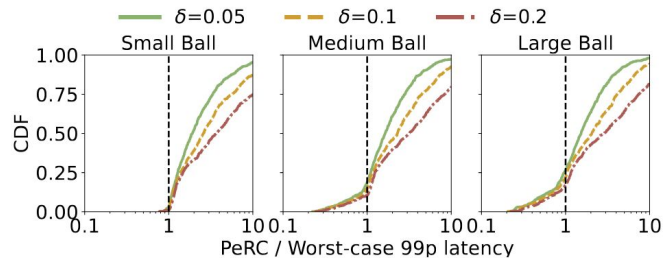
> 1 $\Rightarrow$ PERC is strict upper bound over worst-case latencies

< 1 $\Rightarrow$ weak bound (bad)

Evaluation: PERCs



(a) HR: search Request Type



(b) SN: compose Request Type

Figure 8: Tightness of PERCs: CDF of the ratio of the computed PERC in an environment to the worst-case 99p latency under a perturbation ball. Curves close to the vertical line at 1 imply tight bounds, to the left of the vertical line imply weak bounds, and to the right of the vertical line imply a loose upper bound.

PERC-0.05: provides upper bound over worst-case p99 latency for 93-97% (HR), 96-97% (SN) of evaluated environments

PERC-0.1: provide upper bound for 78-93%

PERC-0.2: provide upper bound for 75-93%

PERCs provide near-full coverage for small perturbations

Evaluation: Galileo-ASc vs. Autothrottle improvements

	HotelReservation	SocialNetwork
Env-A	98.4-99.4%	49-76%
Env-B	75-98%	5-33%

Evaluate end-to-end performance with two environment classes:

- Env-A: varying arrival rates with a mix of request types
- Env-B: varying arrival rates with a mix of request types + CPU-stressing background job

Galileo-ADM vs. TopFull improvements

	Improvement in Avg SLO violations (both benchmarks)
Env-A	0-60%
Env-B	2-46%

Metric: number of SLO violations
SLO = 100ms

Evaluation: Improvement in Controller Efficacy

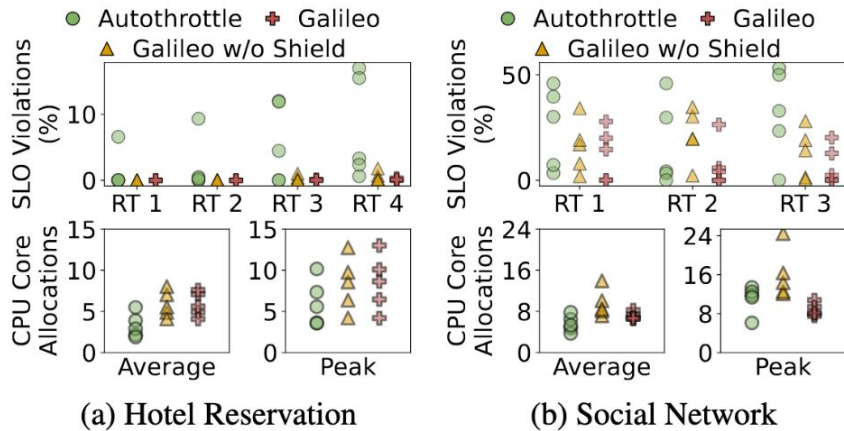


Figure 9: Galileo-ASc vs. Autothrottle on workloads from Env-A. Different RTs stand for various request types (Table 2).

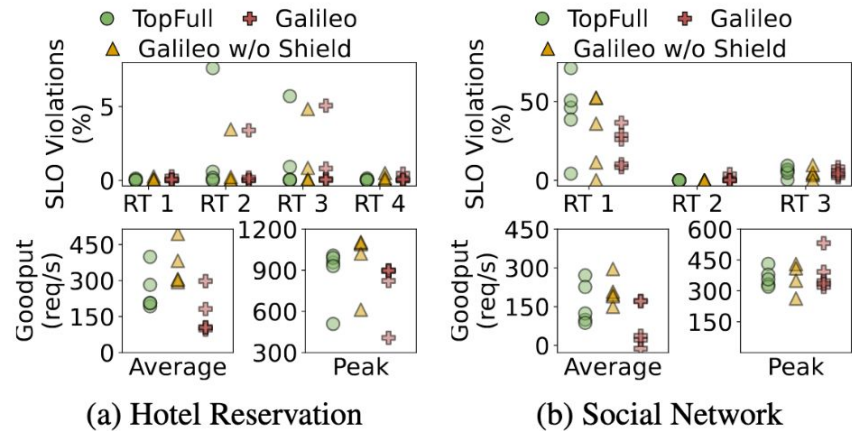
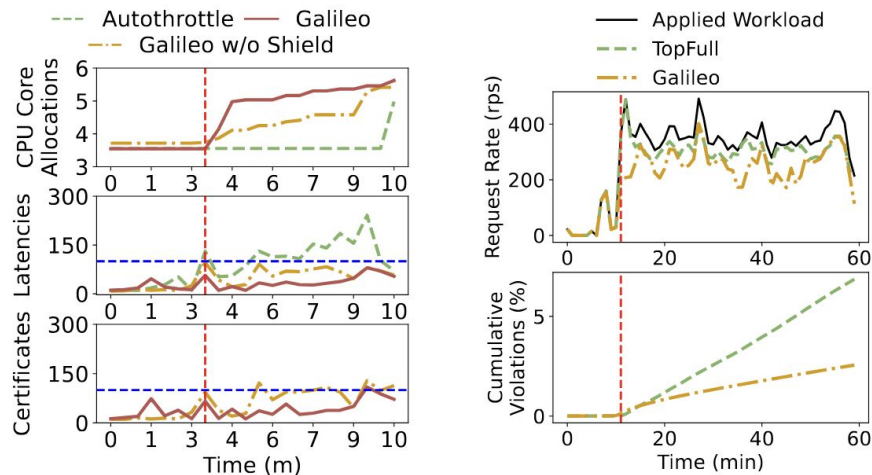


Figure 10: Galileo-ADM vs. TopFull on workloads from Env-A. Different RTs stand for various request types (Table 2).

Evaluation: Improvement in Controller Efficacy



(a) PERCs provide high costs before latency spikes, leading to robust allocations.

(b) Galileo chooses a slightly lower rate limit to be robust to future perturbations.

Figure 11: Deep-dive into the actions of: (a) Galileo-ASc vs. Autothrottle, and (b) Galileo-ADM vs. TopFull, for one trace each - other traces have similar observations.

Galileo performs better due to proactive actions before violations occur

TopFull is not conservative enough in setting rate limit, **Galileo is able to set a more appropriate lower rate limit given current workloads.**

Evaluation: Sensitivity to hyperparameters (PERC computation)

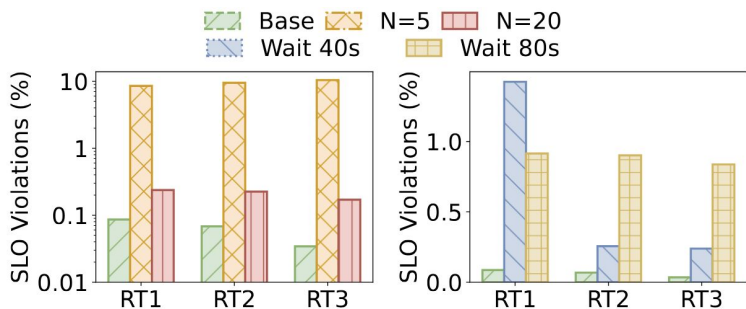


Figure 12: Sensitivity to number of perturbations and wait period.

N = number of perturbations (10)
p = wait period after calculating gradient estimates (20 seconds)

Tradeoff: High N/long p increases accuracy but takes too long to estimate gradient, degrading performance.

Evaluation: Overhead

- Time to fit Gamma parameters (α , β) increases as load increases
- Still, executes w/in 15 ms @ 500 rps
- Computing gradients takes 1.3 ms on average

Negligible compared to total PERC computation cycle (~210 seconds)

Discussion

- What happens when PRM assumptions fail?
 - M/M/1-PS queue
 - Environment remains stable over duration of collecting samples
 - Perturbations are small enough to closely estimate perturbation functions
- How does this approach compare to other autoscaling approaches that are also proactive?
 - [Sinan](#) (ASPLOS '21)
 - [DeepScaler](#) (ASE '23)
 - [AWARE](#) (USENIX '23)